COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Module 25: Asset pipelines, editors and an integrated engine

Expanded Volume 05 / four-lesson module

4 lessons · 20 lesson questions · 5 review questions · Integrating project

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Module 25 — Asset pipelines, editors and an integrated engine

Prerequisites: Module 24 and the software-engineering practices of Module 20.

Planning allocation: 70-110 planning hours, including the project's work and review.

Lesson	Outcome
25.1 — Importing, cooking and content-addressed asset builds	Treat assets as compiled inputs with dependency tracking.
25.2 — Streaming, residency and memory budgets	Load content without allowing asynchronous work to violate ownership.
25.3 — Editors, inspection and undoable commands	Make engine state understandable and safely editable.
25.4 — Engine integration and reproducible demonstration	Deliver a small complete system whose limitations are measurable.

Original module project

Gate E: deliver a small 3D engine demonstration with an asset cooker, reproducible scene, inspector, save/load, diagnostics and a measured frame/memory budget. A second person must build and run it from the README.

Integrating milestones

1. P25-A: reproducible mesh/image cooking with dependency-aware keys.
2. P25-B: bounded loading, cancellation and peak-budget evidence.
3. P25-C: validated inspector commands and undo/redo histories.
4. P25-D: clean build, measurements, second-person review and Gate E defense.

Evidence to retain

- Source/cooked manifest and provenance; cold/warm/change/corruption tests.
- Peak residency ledger; ten-command history tests; raw timings; reviewer record; explicit limitations.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 25.1 / MODULE 25

Importing, cooking and content-addressed asset builds

> **Learning outcome:** Treat assets as compiled inputs with dependency tracking.

Treat content as compiled input

Source assets are convenient for authors; runtime assets are convenient for loading, validation and execution. An importer interprets a source representation. A cooker converts it into the engine's declared runtime representation. A build system decides whether the conversion is necessary and records dependencies. Keeping these roles separate makes a failed asset build understandable and prevents the shipping runtime from depending on every authoring tool. [R20, R26]

Start with a deliberately small source schema: an authored triangle mesh and a tiny RGB image. Validate counts, finite coordinates, channel ranges and schema versions before allocating from untrusted sizes. Declare supported units, coordinate conventions, vertex layout and color semantics. Supporting one representation correctly is more useful than accepting an arbitrary filename and crashing later.

A cache key must describe every relevant input

A content-addressed build key should cover source bytes or a canonical source representation, dependency digests, importer/cooker version, target format and settings. A change to compression, color treatment or coordinate conversion changes the output contract and must invalidate the old result. A timestamp alone is not a trustworthy content identity.

Use an unambiguous encoding for the key description. Raw concatenation can collide structurally: bytes ab followed by c are indistinguishable from a followed by bc unless lengths or structure are encoded. The teaching cooker hashes a canonical JSON description containing named fields and dependency digests. Sorted keys and fixed separators remove irrelevant serialization differences in that description; they do not make every floating-point or source-format conversion universally deterministic.

Dependencies make rebuilds transitive

If a material depends on a texture and the texture changes, the material's build decision must see the updated dependency digest. A mesh may depend on conversion settings or referenced material definitions even when vertex bytes stay identical. Write the dependency graph, reject cycles where unsupported and store a manifest identifying exactly which inputs produced each output.

Separate reproducibility from portability. Identical bytes under the declared cooker/toolchain are a testable target. Different compilers or compression libraries may produce different but valid outputs. Include versions in the record and do not promise bit-identical cross-platform results without evidence.

Publish only complete artifacts

Build into a temporary location, validate the output, then publish it. Keep existing valid artifacts intact when a source is corrupted. A cache hit should verify the expected artifact digest or other appropriate integrity check; the existence of a filename is not proof that its contents are complete. Keep diagnostic messages tied to source path and field or element index.

The included Python cooker uses the standard library and a documented small format, not glTF or a production image compressor. It emits a triangle stream for the C++ CPU renderer and a linear-RGB image mip payload, plus a manifest. Its source image and mesh are original teaching fixtures with a provenance record. The image payload demonstrates cooking and can be inspected by tests; integrating that payload into the runtime texture sampler is a learner task.

Measure the cold and warm paths

Run a clean cook, then an unchanged cook, then a setting change, then a dependency change. Record which keys and outputs change. A malformed-source test must leave a readable error, not a half-valid runtime asset. This is the same engineering discipline used for compiling source code, now applied to the engine's content.

Worked example

Let the key description contain source_sha256, dependency_sha256, cooker_version='ge6-1', target='mesh-f32le-v1' and settings={scale:1.0}. Changing scale to 0.01 changes the key even if the source bytes are identical. Changing an absolute checkout directory should not change it because that path is not a semantic input.

The teaching GEM1 mesh format starts with four magic bytes, a little-endian uint32 vertex count, then eight little-endian f32 values per vertex: x,y,z,u,v,r,g,b. Six vertices use 8+6*32=200 bytes. The loader rejects a count not divisible by three, truncated bytes, extra unexpected payload and nonfinite fields.

An unchanged build in two fresh output directories should produce the same artifact bytes and manifest under the tested environment. That demonstrates the declared fixture's reproducibility; it does not prove that every future importer will behave identically.

Original lab assignment

Build a cooker for one mesh and one image representation. Pass: identical declared inputs reproduce the same result, changed settings invalidate the cache, and corrupted sources produce readable failures.

Assigned study

R20: asset pipelines; R26: asset management and content tools. Apply Module 20 dependency and provenance practices.

Focused study: R20/R26: asset pipeline roles. S03: compare a real interchange schema to the small teaching schema. Reuse Module 20 provenance and dependency practices.

Check your understanding

1. Why distinguish source assets from runtime assets?
2. Must a compression-setting change invalidate a cached build?
3. Why include tool version in a cache key?
4. Why is hashing raw concatenation of variable-length input blobs risky without framing?
5. How many bytes does a GEM1 asset with six vertices occupy under the stated eight-f32-per-vertex layout?

LESSON 25.2 / MODULE 25

Streaming, residency and memory budgets

> **Learning outcome:** Load content without allowing asynchronous work to violate ownership.

Loading is a state machine with ownership

An asynchronous request should move through explicit states such as requested, reading, decoding, waiting-for-upload, uploading, resident, retiring, cancelled and failed. Each state has an owner and a permitted resource set. The renderer uses a stable placeholder until the required representation is resident. A file read completing does not mean a usable GPU texture exists. [R20, R26]

Separate the request identity from the resource identity. A handle names the desired asset; a ticket identifies this particular load attempt. Reload, scene change or cancellation can make a completion stale. Check a generation or revision token before publishing the result. Late completions must release their temporary allocations without reviving an unwanted asset.

Bound every expensive stage

A small compressed file can expand into a much larger decoded image. During transition, memory can include compressed input, decoded CPU bytes, staging/upload storage, new GPU resident bytes and an old generation waiting for retirement. Count peak overlap, not merely steady-state resident payload.

Maintain separate budgets when memory pools are separate, such as CPU decode and GPU device-local memory. A combined budget model is useful for a simplified exercise but must not be mislabeled as actual hardware accounting. On unified-memory systems, logical CPU/GPU payload counters can double-count shared allocations unless their ownership is understood. Record the estimator's assumptions.

Limit queue depth and in-flight decoded bytes, not just the number of worker threads. Two workers each holding several enormous decoded assets can exceed a memory budget despite a small thread pool. Admission should reserve the expected peak cost before beginning a stage or choose a fallback when an estimate is unavailable.

Cancellation is not magic reclamation

A cancellation flag can prevent future stages or publication, but it may not interrupt an operating-system read or a submitted GPU upload. Keep the allocation until the active consumer finishes, then retire it. Never free upload storage simply because the user changed scenes. A state machine should distinguish cancelled-but-in-flight from fully reclaimed.

Eviction also has conditions. An asset may be logically unused by the world but still referenced by a submitted frame. Use reference or pin counts for current owners and completion tracking for asynchronous users. Avoid evicting a fallback asset that every missing texture relies on.

Budget failure is an ordinary case

Force a tiny budget so the test can reliably exercise rejection and fallback. Select lower-resolution content, defer an optional request, evict eligible content or retain the placeholder according to an explicit priority policy. Essential content can fail the scene load with an actionable error. An endless retry loop that repeatedly decodes an impossible asset is not graceful degradation.

Record request count, rejected admissions, pending bytes, peak decoded bytes, resident bytes and retirement backlog. Add a cancellation test immediately before completion. The teaching budget and state

models help reason about these cases; the complete multithreaded streaming subsystem and GPU-residency measurement remain lab work.

Worked example

Assume an explicitly combined teaching budget of 100 MiB. The engine holds 40 MiB resident data and 20 MiB of existing temporary/retiring data. A new request needs 30 MiB decoded staging plus 20 MiB of new resident data at peak. Its estimated peak is 40+20+30+20=110 MiB, so admission fails even though final resident data would be only 60 MiB after cleanup.

The valid response may be to finish retiring the 20-MiB old allocation, then admit the request at a 90-MiB peak. It is invalid to free that old allocation early while an in-flight command still references it.

Request ticket 12 is cancelled while upload is submitted. Completion can move it to retired, releasing staging after the consumer finishes, but cannot publish it as resident. A newer request ticket 13 for the same asset is a separate lifetime and must not inherit ticket 12's cancellation or completion accidentally.

Original lab assignment

Implement a bounded streaming queue with placeholder assets and cancellation. Pass: a forced small budget produces graceful fallback, no use-after-free, and a report of peak rather than only steady-state memory.

Assigned study

R20: resource loading; R26: asset management, asynchronous loading and streaming concepts.

Focused study: R20/R26: asynchronous loading and streaming. S05/S07: retirement follows actual consumers. Keep the memory estimator distinct from device measurement.

Check your understanding

1. Why is compressed file size not the complete memory budget?
2. What should rendering do while an optional asset is unavailable?
3. Why model loading as explicit states?
4. Under the example 100-MiB budget, why is a request ending at 60 MiB resident still rejected?
5. Does cancelling an already submitted upload make its staging allocation immediately reusable?

LESSON 25.3 / MODULE 25

Editors, inspection and undoable commands

> **Learning outcome:** Make engine state understandable and safely editable.

An inspector is a controlled view of the model

An editor makes engine state inspectable without becoming a second inconsistent copy of the world. The model owns authoritative entity and asset data. The view presents that data and captures user intent. A controller or command layer validates changes, applies them at a safe boundary and reports failure. A text inspector is acceptable for this small engine; correctness matters more than widget count. [R20, R26]

Show stable identity, transform values, component presence and relevant resource status. Distinguish authoring state from transient simulation state. Editing a simulated position may update an authoring transform, teleport a live body or be disallowed while playing; choose and label the behavior. Do not silently save transient physics output as the source scene.

Commands make changes reversible

Represent a committed transform edit as a command containing entity identity, before value and after value. Apply validates the current target and new value; undo restores the before value under the appropriate conflict policy. Redo reapplies the after value. A new edit after an undo invalidates the redo branch unless the editor deliberately supports branching history.

A mouse drag can emit hundreds of previews but should usually commit one meaningful command. Save the initial value when the drag begins, update a temporary preview while dragging, and push a single before/after command on commit. Cancel discards the preview. This distinction keeps undo useful and prevents partially edited state from becoming a durable save.

Validation belongs below the UI

Reject nonfinite numbers, missing entities, invalid scale ranges under the chosen model and unsupported operations. A text box can be bypassed by a script or file import; validate at the command/model boundary as well. A negative scale may be valid if the engine handles reflected winding and tangent frames, but it should not be accidentally accepted by a renderer that assumes only positive scales.

Undo is a state transition, not an exemption from validation. The target entity may have been deleted or reloaded. Define how stable IDs resolve and how incompatible history is cleared. In this teaching engine, a scene load clears history because commands refer to the previous scene revision. A richer editor may instead store explicit revision-aware commands.

Input focus prevents unintended actions

Keyboard input intended for a numeric field must not simultaneously move the camera or trigger gameplay. Route events through a declared focus/capture policy. Pressing a gameplay key while a field has focus should edit or be ignored according to the UI, not issue an unrelated world command. Losing focus during a drag needs a commit or cancel policy.

Test histories, not only buttons

Create ten transform commands, undo all ten, redo all ten and compare exact logical state. Then undo three and make a new edit; verify redo is empty. Attempt an invalid command and ensure neither world state nor

history changes. Save, reload and compare committed values. The supplied command-history helper demonstrates the data model; the full inspector, focus integration and engine coupling remain required practical work.

Worked example

Entity lamp has position (0,0,0). A committed move stores before=(0,0,0), after=(2,0,0). Undo restores the former; redo restores the latter. Three previews at x=0.5,1.0,2.0 during one drag still produce just one command when committed.

After commands A,B,C, undo C and B. The model is at A and the redo stack can restore B then C. Commit new command D now: the correct ordinary linear history is A,D, and the redo branch containing B,C is cleared. Keeping it would allow replaying commands whose assumed before-state is no longer the current history.

A request to set x to NaN is rejected before mutation. Verify both the entity transform and the lengths of undo/redo history are unchanged. The absence of a crash is not sufficient; the command must be transactionally rejected.

Original lab assignment

Create a scene inspector with transform editing and a ten-command undo/redo test. Pass: invalid inputs are rejected, focus does not trigger unintended gameplay, and save/reload preserves committed changes.

Assigned study

R20: game tools; R26: editor and reflection concepts. Use a minimal UI toolkit appropriate to the selected platform.

Focused study: R20/R26: tools and model inspection. R11: command interface contracts. Apply Module 20 input-validation and transactional-state habits.

Check your understanding

1. What is the model/view distinction useful for?
2. What data can make a transform move reversible?
3. Why distinguish preview state from committed authoring state?
4. After undoing two commands, what should a new edit do to an ordinary linear redo stack?
5. Why should a drag preview not normally be written directly into the saved authoring scene before commit?

LESSON 25.4 / MODULE 25

Engine integration and reproducible demonstration

> **Learning outcome:** Deliver a small complete system whose limitations are measurable.

Integration proves the boundaries agree

Gate E is a small complete engine demonstration, not a collection of disconnected attractive screenshots. Its CPU reference renderer, selected GPU route where available, fixed-step world, entity/resource ownership, asset cooker, inspector, versioned saves and diagnostics must agree about units, identity, coordinates and lifetime. The project can be visually simple. A few meshes, a controlled light and a moving entity are enough to expose real architecture problems.

Preserve a headless world path so tests run without a display. Keep a fixed reference scene with original or properly attributed assets, a deterministic input sequence under the declared reproducibility scope, and a saved camera. Build and cook commands should work from a clean extracted tree, not depend on an absolute path or a file that exists only in a developer's cache.

Budget against a declared machine

Choose resolution, quality settings, scene contents and a test environment. A 60-fps target corresponds to about 16.667 ms per presented frame, but it is an example budget, not a universal passing score. A CPU reference renderer may have a much slower target and still provide valuable correctness evidence. State separate budgets for CPU simulation/recording, GPU execution when measured, full-frame pacing and memory.

Record warm-up, sample count, timer scopes and raw samples. Report median, p95, p99 and maximum for the selected workload, using a stated percentile definition. Never relabel a CPU timer around submission as GPU time. API timestamp queries require their own support, units and synchronization. Separate initialization and asset-cook time from steady-state frame time.

Use adversarial integration scenes

Test a triangle crossing the near plane, rapid resize/minimize/restore, stale handles, malformed assets, cancelled loads, save migration, invalid editor input and shutdown with pending work. Failure injection should have a predictable expected state. An application that survives by silently disabling half its features needs that quality change in the report.

Image comparisons require a known reference, matching camera, extent, sample convention and color domain. The supplied CPU renderer's integer-quantized coverage can differ from hardware at edges. Compare interiors with an appropriate tolerance and separately inspect edges; do not use a broad tolerance to hide a flipped image or incorrect UV interpolation.

Reproducibility needs another set of hands

The canonical checkpoint requires a second person to build and run the engine from its README. A clean-directory automated test is useful but not the same evidence. Give the reviewer the source tree and commands, record what they did, and fix every undocumented assumption. Ask them to reproduce one boundary test and one live edit, then inspect a save/reload result.

The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. A reference implementation copied without understanding does not pass the oral review. Existing engines may be used for comparison, but cannot replace the central implementation and still meet this checkpoint's objective.

Close with a limitations record

Mark capabilities implemented, tested, pending or deliberately out of scope. The bundle itself includes educational components and reference tests; it does not certify the learner's Gate E engine or claim a hardware run not performed. A trustworthy limitations section makes the next volume possible: Game Systems and Production builds on the engine's actual contracts, not its aspirational feature list.

Worked example

At 60 frames per second, the ideal interval is 1000/60=16.6667 ms. If 99 frames take 10 ms and one takes 200 ms, the mean is 11.9 ms, which sounds comfortably below the budget. Yet the 200-ms stall remains visible. Under nearest-rank percentiles, p95=10 ms, p99=10 ms and maximum=200 ms for this particular 100-sample set. This demonstrates why a maximum, hitch count and sufficient sample count matter even when percentiles are reported.

For ten sorted samples [8,8,9,9,10,10,11,12,18,40], nearest-rank p95 is the element at ceil(0.95*10)-1, which is index 9: 40 ms. State the definition so another implementation can reproduce the number.

A clean build in an empty directory can reveal a missing generated header or uncooked asset. A second-person build can additionally reveal an ambiguous instruction such as run the engine without saying which working directory or executable. Both tests have value, but they answer different questions.

Original lab assignment

Complete Gate E using the project specification in the handbook. Pass: another person follows the README, a headless test scene passes, all assets have provenance, and a limitations section identifies remaining correctness or performance risks.

Assigned study

R20: project completion; R26: profiling and engine architecture comparison; revisit Module 20 release and review practices.

Focused study: Canonical Gate E specification; R20/R26: engine integration and profiling. Revisit Module 20 clean-build, provenance and review requirements.

Check your understanding

1. What frame interval corresponds to 60 frames per second?
2. Why report more than average frame time?
3. What does a clean-machine build test reveal?
4. For 99 frames at 10 ms and one at 200 ms, compute the mean and explain why it hides a problem.
5. Why is a clean-directory automated build not sufficient to mark the second-person review complete?

Module 25 review

1. Which key inputs prevent a stale content build after a tool upgrade?
2. Why can bounded worker count still exceed the memory budget?
3. What must happen to editor history when its target scene revision is replaced?
4. What does the software-only Gate E allowance exclude?
5. What must a reviewer reproduce besides opening a screenshot?

Project handoff

Retain the gate e integrated engine evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Reading references: IDs retain their meanings in source/REFERENCES.md and the full handbook. Use those links alongside the lesson-specific reading assignments.